쇼핑 AI Agent · 한 줄 정의

내가 자주 입는 옷을 기준으로, 비슷한 옷을 근거와 함께 찾아 비교해 주는 AI 에이전트

구매 이력이 아니라, **'왜 그 옷이 좋은지'** 를 기준으로 옷을 찾습니다.

Why now

옷을 고르는 일은, 여전히 '사람의 몫'입니다

- **많은 쇼핑몰 추천** 은 최근 본 상품과 구매 이력을 주요 기준으로 사용합니다.
- 핏·소재·사이즈·실루엣은 결국 **제가 하나씩 비교**합니다.
- '마음에 드는 이유'는 **말로 옮기기 어려워** 검색 기준으로 사용하기 어렵습니다.
- 착용감·실용성을 따질수록 **시간이 더 걸립니다.**

필요성

좋아하는 옷의 **'이유'를 기준으로 찾고 비교하는 과정**을 대신할 도구가 필요합니다.

How it works for you

자주 입는 옷을 보여주면 , 에이전트가 후보를 찾고 비교합니다

01

보여주기

자주 입는 옷을 사진이나 말로 설명합니다. "이 차콜 반팔티 핏이 좋아요."

02

찾고 비교하기

에이전트가 후보를 탐색하고, 상세페이지와 이미지에서 확인 가능한 근거를 수집 합니다.

03

받기

근거가 붙은 후보 3~5개와 추가 확인이 필요한 항목을 비교표로 받습니다.

사진 자연어 구매처 힌트 → 옷 용어를 몰라도, 평소 말투로 요청하면 됩니다

The output

받는 것은 추천 문장이 아니라, 판단할 수 있는 후보 카드

- 왜 비슷한지, 무엇이 다른지가 **한눈에** 보입니다.
- 사이즈 리스크와 **근거(출처)**가 함께 붙습니다.
- **구매 전에 후보를 비교하고 판단**할 수 있는 자료입니다.

상품·링크 | 유사한 이유 | 차이점 | 사이즈 리스크 | 근거 | 누락 정보



The decision pipeline · 한 마디가 후보가 되기까지

“차콜 반팔티 핏이 좋아요” — 이 한마디를 속성으로 쪼개 검색·판단합니다

01 요청 이해 `parse_user_goal`

속성 추출 [반팔티] [박시핏] [총장 길게] [탄탄한 면] · **우선순위 핏 > 내구성 > 디자인** · 색은 고정 조건 아님

02 검색어 구성 `search_web_candidates`

속성 → 쿼리 [“오버핏 반팔티 차콜”] [“탄탄한 면 박시한 반팔”] [“20수 면 오버 티”] · 연관 표현 확장: 박시 · 오버핏 · 세미오버핏

03 후보 선별 `filter + rank`

검색 후보 → [~~카테고리–불일치~~] [~~얇은–소재~~] [~~슬림–실루엣~~] 제외 → **비교 후보** · 제외 사유도 기록

04 근거 기반 비교 `score_candidate_match`

핏 · 소재 · 실측 · 디자인 근거와 누락 정보로 적합도를 판단 · 근거 부족 시 [need_more_info 보류]

→ 맞는 이유뿐 아니라, **제외한 이유와 확인이 필요한 정보까지** 함께 보여줍니다.

Context, Memory & State Architecture

문맥과 상태를, 한 번의 프롬프트에 몰지 않고 나눠 관리합니다

이번 작업의 문맥

Run Context · 단기 기억

- 현재 목표와 검색 조건
- 기준 옷과 후보 상품
- 아직 해결되지 않은 정보
- 최근 질문과 사용자 답변

`run_id` 단위로 저장하고, 매 단계가 끝날 때 갱신

사용자의 취향

Closet Memory · 장기 기억

- 자주 입는 옷
- 선호 핏 · 소재 · 실루엣
- 피하고 싶은 디자인
- 과거 비교와 피드백

사용자 단위로 분리하고, 필요한 항목만 현재 run에 불러옴

판단 근거

Evidence Store · 관찰 기록

- 검색 결과와 상품 URL
- 상세페이지 추출 정보
- 이미지 추출 결과와 신뢰도
- 점수 · 누락 필드 · 출처

원문 전체가 아니라 **요약과 출처 참조**만 문맥에 전달

Backend Runtime FastAPI가 도구 실행과 상태 갱신을 소유 | 이미지·원문은 별도 저장, DB엔 참조값 | 자격 증명·provider key는 모델 밖에서 관리

`GET /runs/{run_id}` 현재 상태 조회 **`SSE /runs/{run_id}/events`** 실행 단계·trace 실시간 확인

Goal-Driven Agentic Loop

정해진 순서가 아니라, 상태를 보고 다음 도구를 고릅니다

1 **Goal** 무엇을 찾고, 무엇을 기준으로 비교할지 정의

2 **State 확인** 지금까지 확인한 사실과 부족한 정보를 읽음

3 **행동 선택** 현재 상태에 필요한 다음 도구를 LLM이 선택

4 **도구 실행** Backend가 schema를 검증하고 실제 기능을 실행

5 **Observation** 결과를 사실·누락·실패·출처로 저장

6 **다음 결정** `completed · waiting_for_user · failed`

↳ 목표를 못 채우면 2. State 확인으로 되돌아가 반복

TOOL REGISTRY

`name + description`

`JSON schema`

`timeout / retry`

`risk class`

`structured output`

Understand Search Inspect Extract Compare Report

상태와 결과에 따라 사용할 도구 선택

설계

Recovery, Guardrails & Observability

실패해도 근거 없이 답하지 않는 구조

문제	먼저 시도하는 복구	최종 상태
검색 결과 없음	검색어 확장 · 다른 provider	재탐색 또는 failed
상세페이지 접근 실패	retry · 다른 fetch 경로	실패 observation 저장
사이즈·소재 누락	fetch → 이미지 추출 → 정규화	그래도 없으면 waiting_for_user
잘못된 tool 인자	schema 오류를 모델에 반환	수정된 인자로 재호출
같은 실패 반복	반복 횟수·인자 비교	guardrail로 중단
근거 없는 최종 문장	grounding validator가 거부	재작성 또는 추천 보류

복구 순서

Retry → Alternative → Ask User → Stop Safely

상태 전이

created → running ⇄ waiting_for_user → completed

Verified Scope & Next · 2026-06-09 재검증

설계를 어디까지 검증했고 무엇을 이어갈지 구분했습니다

구현 · 실제 확인

LLM이 상태를 보고 다음 도구를 선택 하는 agent loop

backend-owned search·fetch·OCR·normalize·score

run·observation·trace SQLite 저장

SSE 상태 확인 · 사용자 추가 답변

Brave live search · Tesseract 실행

observation 기반 report grounding gate

조건부

상품 페이지별 실측·소재 확보

OCR의 usable text 추출

최종 구매 report 완료

로컬 LLM 실사용

연결은 됨 · 성공률은 provider·상품 페이지에 따라 달라짐

향후 구현 방향

사진 기반 핏·실루엣 vision 이해

쇼핑몰별 실측·소재 추출 안정화

사용자별 장기 closet memory

코디·스타일 단위 추천

실제 LLM과 live search/OCR가 연결된 agent runtime prototype까지 구현 — 핵심 근거를 못 채우면 추천 대신 **사용자에게 추가 정보를 요청** 합니다.

향후 계획

Roadmap to a Personal Shopping Agent

한 번을 잘하는 도구에서, 기준을 기억하는 에이전트로

Phase 1
근거 수집 안정화
쇼핑몰별 fetch adapter • OCR/vision 품질 개선 • 상품 spec confidence 강화

Phase 2
개인화 메모리
closet memory 자동 갱신 • 구매·반려 피드백 저장 • 상황별 핏·소재 선호 학습

Phase 3
스타일 에이전트
한 벌 → 코디 단위 추천 • 계절·장소·예산 고려 • 만족도를 memory에 재반영

한 번의 추천을 잘하는 도구에서, **사용자의 기준을 기억하고 다음 선택까지 더 잘 돕는** 에이전트로 확장합니다.